

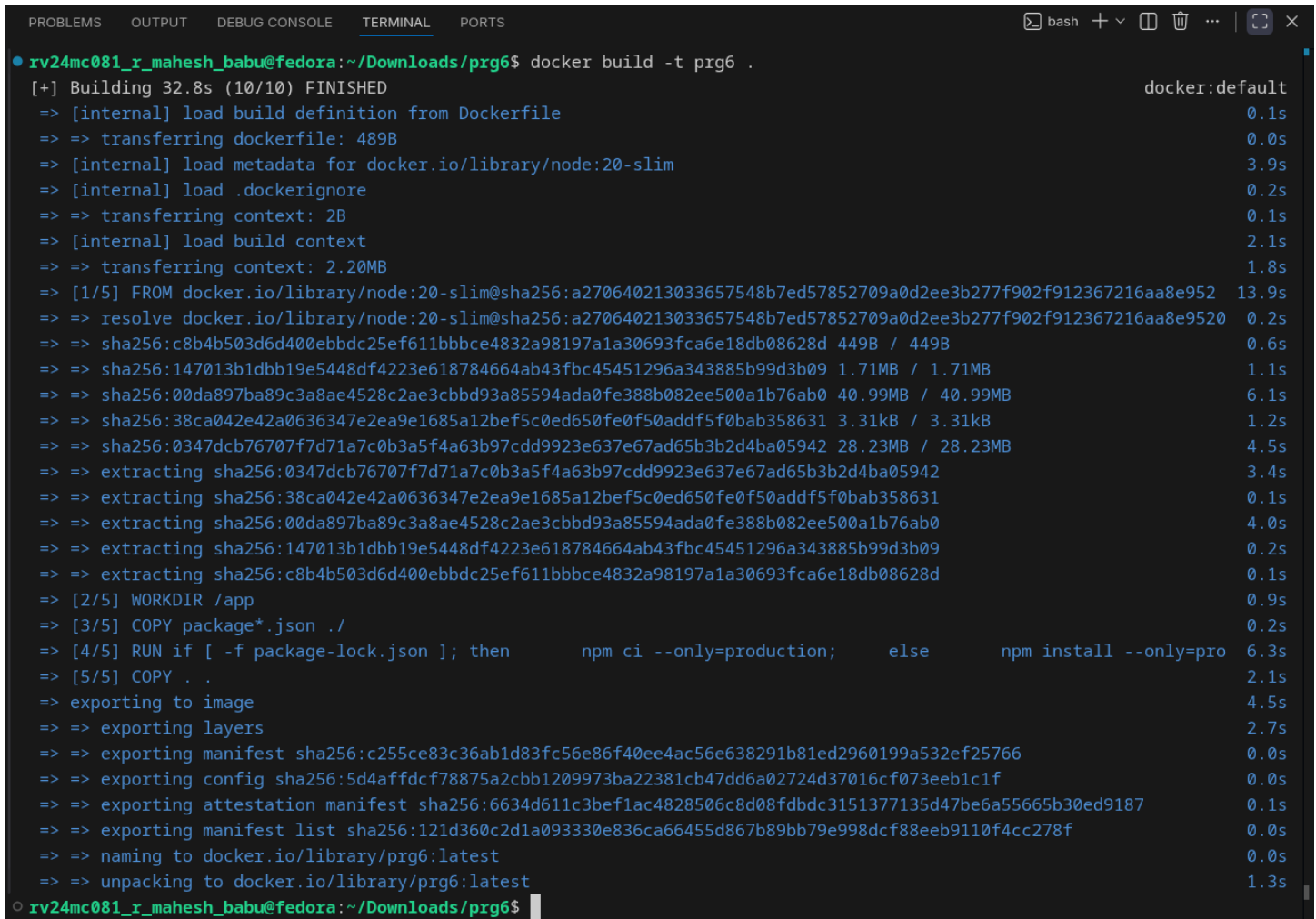
Program-6

Title: Implementing an Automated Build Pipeline Using Docker Hub

This document describes the step-by-step process of developing an application, containerizing it with Docker, and publishing the resulting image to Docker Hub and the GitHub Container Registry (GHCR). This workflow establishes a basic yet effective CI/CD pipeline.

1. Local Project Setup

The process begins with creating the project directory and preparing the local development environment. This step includes organizing files and initializing the application source code required for containerization.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$ docker build -t prg6 .
[+] Building 32.8s (10/10) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 489B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-slim    3.9s
=> [internal] load .dockerignore                                  0.2s
=> => transferring context: 2B                                       0.1s
=> [internal] load build context                                  2.1s
=> => transferring context: 2.20MB                                    1.8s
=> [1/5] FROM docker.io/library/node:20-slim@sha256:a270640213033657548b7ed57852709a0d2ee3b277f902f912367216aa8e952 13.9s
=> => resolve docker.io/library/node:20-slim@sha256:a270640213033657548b7ed57852709a0d2ee3b277f902f912367216aa8e9520 0.2s
=> => sha256:c8b4b503d6d400ebbd25ef611bbbce4832a98197a1a30693fca6e18db08628d 449B / 449B 0.6s
=> => sha256:147013b1dbb19e5448df4223e618784664ab43fbc45451296a343885b99d3b09 1.71MB / 1.71MB 1.1s
=> => sha256:00da897ba89c3a8ae4528c2ae3cbbd93a85594ada0fe388b082ee500a1b76ab0 40.99MB / 40.99MB 6.1s
=> => sha256:38ca042e42a0636347e2ea9e1685a12bef5c0ed650fe0f50addf5f0bab358631 3.31kB / 3.31kB 1.2s
=> => sha256:0347dcb76707f7d71a7c0b3a5f4a63b97cdd9923e637e67ad65b3b2d4ba05942 28.23MB / 28.23MB 4.5s
=> => extracting sha256:0347dcb76707f7d71a7c0b3a5f4a63b97cdd9923e637e67ad65b3b2d4ba05942 3.4s
=> => extracting sha256:38ca042e42a0636347e2ea9e1685a12bef5c0ed650fe0f50addf5f0bab358631 0.1s
=> => extracting sha256:00da897ba89c3a8ae4528c2ae3cbbd93a85594ada0fe388b082ee500a1b76ab0 4.0s
=> => extracting sha256:147013b1dbb19e5448df4223e618784664ab43fbc45451296a343885b99d3b09 0.2s
=> => extracting sha256:c8b4b503d6d400ebbd25ef611bbbce4832a98197a1a30693fca6e18db08628d 0.1s
=> [2/5] WORKDIR /app                                             0.9s
=> [3/5] COPY package*.json ./                                    0.2s
=> [4/5] RUN if [ -f package-lock.json ]; then npm ci --only=production; else npm install --only=pro 6.3s
=> [5/5] COPY . .                                                 2.1s
=> exporting to image                                             4.5s
=> => exporting layers                                             2.7s
=> => exporting manifest sha256:c255ce83c36ab1d83fc56e86f40ee4ac56e638291b81ed2960199a532ef25766 0.0s
=> => exporting config sha256:5d4affdcf78875a2cbb1209973ba22381cb47dd6a02724d37016cf073eeb1c1f 0.0s
=> => exporting attestation manifest sha256:6634d611c3bef1ac4828506c8d08fdbdc3151377135d47be6a55665b30ed9187 0.1s
=> => exporting manifest list sha256:121d360c2d1a093330e836ca66455d867b89bb79e998dcf88eeb9110f4cc278f 0.0s
=> => naming to docker.io/library/prg6:latest                     0.0s
=> => unpacking to docker.io/library/prg6:latest                 1.3s
● rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$
```

2. Version Control and Source Code Management (Git & GitHub)

The application source code is tracked using Git and pushed to a remote GitHub repository. This enables version control, collaboration, and serves as the foundation for future automated build processes.

3. Building the Docker Image Locally

In this stage, the application along with its runtime environment—defined within the Dockerfile—is packaged into a single, executable Docker image. This image contains all dependencies and configurations needed to run the application consistently across different systems.

Login with Dockerhub credentials and with github credentials

```
rv24mc081_r_mahesh_babu@fedora:~$ docker login

USING WEB-BASED LOGIN

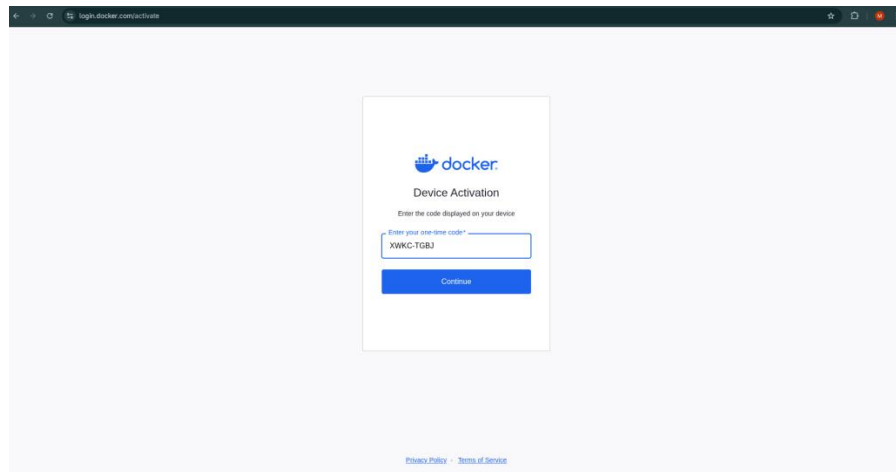
Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: XWKC-T6BJ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/home/rv24mc081_r_mahesh_babu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
rv24mc081_r_mahesh_babu@fedora:~$
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$ docker push mahesh158/prg6:latest
The push refers to repository [docker.io/mahesh158/prg6]
0347dc76707: Pushed
6aa6c21511ff: Pushed
7b35c2726f0b: Pushed
c8b4b503d6d4: Pushed
8369cc62f7e5: Pushed
d8d1bb2199c7: Pushed
38ca042e42a0: Pushed
00da897ba89c: Pushed
147013b1d6b1: Pushed
1e882b85561c: Pushed
latest: digest: sha256:121d360c2d1a093330e836ca66455d867b89bb79e998dcf88eeb9110f4cc278f size: 856
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$ docker tag prg-6 ghcr.io/R Mahesh Babu/prg6:latest
```

```
uuJfMW5nMc3WVpBV | ghcr.io -u /rvcestudent --password-stdin

WARNING! Your credentials are stored unencrypted in '/home/hesh Babu-p/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$ echo ghp_2Vf4d2p3b1SYjj
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg6$ docker push ghcr.io/rvcestudent/program-6:latest
```